

一周一课 · AI Infra 第一期

大模型这么火 AI Infra 到底是干什么的? 值不值得入行?

从你写的一行训练代码，到几千张卡跑起来
中间到底发生了什么

时长 ≈ 40 分钟

面向：想入行 AI Infra 的新人

讲师：就业培训 · 一周一课



这 40 分钟，我们回答 5 个问题

Q1 · 是什么

AI Infra 到底干什么

一行代码 → 几千张卡，中间发生了什么

Q2 · 有哪些岗

6 大方向，新人从哪切入

训练 / 推理 / 集群调度 / 存储数据 / 框架算子 / 稳定性

Q3 · 考什么

面试高频考点

OS、网络、并行、C++/Python、Linux 排障

Q4 · 练什么

没集群也能做的项目

把简历从"调包"做成"懂系统"

最后一个问题 **Q5**：3-5 年后，这个岗位是被替代，还是更稀缺？入行前 3 年该怎么积累。



模块一 / 5

AI Infra 是什么

从你写的一行训练代码
到几千张卡跑起来，中间到底发生了什么

你写的其实只有一行: `model.train()`

但让这一行"在几千张 GPU 上真正跑起来、跑得快、跑不崩"的那一大坨东西, 就是 AI Infra。

写训练代码



数据读取喂给 GPU



模型切分到多卡多机



卡间高速通信同步



调度到集群 / 容错重启

算法同学看到的

Loss 在下降, 模型在变好

Infra 同学看到的

显存、带宽、通信、故障、利用率

老板看到的

几千张卡的电费有没有白烧



AI Infra = 让 AI 算力"高效、稳定、规模化"跑起来的系统层

高效

让昂贵的卡不闲着

提高 GPU 利用率 / 训练吞吐，同样的卡多跑 30% 就是省钱

稳定

千卡跑几周不崩

一张卡挂了，几千张卡的任务不能全部白跑

规模化

从 8 卡到万卡

单机能跑，扩到成百上千台机器还能跑得动

类比：算法是**开车的人**，AI Infra 是**发动机、变速箱、路和加油站**。车再好，没有这套东西也跑不起来。



大模型时代，Infra 从"支撑"变成了"胜负手"

模型越大，Infra 越重

- ◆ 单卡装不下，必须多卡多机切分
- ◆ 训练一次几百上千万成本，效率差一点就是烧钱

算力是稀缺资源

- ◆ 卡贵、电贵、机房贵
- ◆ 谁能把利用率做上去，谁就有竞争力

推理决定能不能赚钱

- ◆ 模型上线后，成本天天在发生
- ◆ 推理优化直接影响毛利

人才供给远小于需求

- ◆ 懂系统又懂 AI 的人本来就少
- ◆ 门槛高 → 稀缺 → 值钱



模块二 / 5

AI Infra 有哪些岗位方向

6 个方向全景 · 哪些更适合新人当第一份工作切入

AI Infra 的 6 大方向

01 训练

Training

分布式训练、并行策略、显存优化，让大模型训得起来、训得快

02 推理

Inference

模型上线服务、延迟与吞吐优化、量化、KV Cache，直接影响成本

03 集群/调度

Cluster & Scheduling

成百上千张卡怎么分配、排队、抢占，让资源不空转

04 存储与数据

Storage & Data

海量训练数据的存储、读取、预处理，别让 GPU 等数据

05 框架与算子

Framework & Kernel

PyTorch 底层、CUDA 算子、编译优化，压榨硬件性能

06 稳定性

Reliability

监控、容错、故障定位，千卡任务跑几周不崩



哪些方向对新人更友好？

方向	核心能力	入门门槛	新人友好度
训练	并行/分布式/显存	高	★★☆
推理	服务化/性能优化	中	★★★★
集群/调度	K8s/资源管理/OS	中	★★★★
存储与数据	IO/数据管线/工程	中低	★★★★★
框架与算子	C++/CUDA/编译	很高	★★☆
稳定性	监控/排障/自动化	中低	★★★★

规律：越靠近工程、越有明确产出的方向，越适合新人先站稳脚跟；算子/训练这类"硬核"方向，适合作为进阶目标。

新人第一份工作，建议这样切入

首选切入点

数据/存储 · 稳定性 · 推理服务

- ◆ 产出看得见（数据管线跑通、服务上线、故障变少）
- ◆ 需要的是扎实工程能力，不是天才级底层功底
- ◆ 能快速接触真实集群和真实问题

进阶目标

训练 · 框架与算子

- ◆ 在真实系统里泡过之后再往下钻
- ◆ 底层功底 + 项目积累后回报最高
- ◆ 不建议新人一上来就死磕 CUDA 算子

一句话：先用工程能力上牌桌，再用系统深度往上走。



模块三 / 5

面试到底考什么

高频基础能力 + 怎么把你的项目讲成"懂系统"

招聘里反复考察的 5 块基础

01 操作系统

进程/线程、内存、调度、IO

为什么显存会 OOM、为什么进程卡住、页缓存是什么

02 网络

TCP/IP、带宽/延迟、RDMA 概念

多机训练为什么慢，瓶颈在计算还是通信

03 并行计算

数据/模型/流水线并行

一个大模型怎么切到多卡上，通信怎么发生

04 C++ / Python

Python 熟练 + C++ 能读能写

脚本和工具用 Python，性能关键路径看 C++

05 Linux 排障

命令行 + 定位问题的能力

top/htop/nvidia-smi/strace/日志，能不能自己把问题定位到根因，是区分"调包侠"和"工程师"的分水岭



面试官其实只想确认三件事

基础扎不扎实

会不会写代码、懂不懂系统

用 OS/网络/并行的问题快速探底

会不会排障

遇到问题怎么定位

"任务卡住了你怎么查？"——看思路，
不看背答案

项目是否真做过

是调包还是懂原理

顺着你的项目往下追 3 层，看你能不能
答上来

结论：背八股能过初筛，但决定 offer 的是"你有没有真在系统里动过手、踩过坑"。

把"我做过"讲成"我懂系统"

☒ 调包式讲法

- ◆ "我用 PyTorch 训了个模型"
- ◆ "我调了下参数，效果还行"
- ◆ 没有数字、没有瓶颈、没有权衡

☒ 懂系统式讲法

- ◆ 训练卡在 IO，我把数据预处理改成流水线，GPU 利用率从 40% 提到 75%
- ◆ 多卡通信是瓶颈，我换了并行策略，单步耗时降了 30%
- ◆ 有指标、有定位、有取舍

讲项目公式：**遇到什么问题** → **怎么定位** → **怎么解决** → **量化结果** → **学到了什么**。哪怕是小项目，讲清这条链路就赢了一半。



模块四 / 5

没集群，新人能做什么

没有几千张卡、没有生产环境
照样把简历从"调包"做成"懂系统"

规模买不到，"系统思维"练得出来

原则一

用小规模复现大问题

2 张卡也能练分布式，1 台机器也能练
调度和排障

原则二

盯住指标，不盯效果

关注吞吐、延迟、利用率、显存，而不
是模型准确率

原则三

把过程写下来

踩坑、定位、优化写成文档/博客，就是
最好的项目背书

面试官不在乎你有没有万卡，只在乎你面对系统问题时的思路 and 手感。

4 个不用集群就能做的项目

项目	用什么	练到什么能力
单机多卡/单卡多进程分布式训练	PyTorch DDP / 免费云薅羊毛	并行、通信、进程管理
推理服务部署 + 压测	vLLM/FastAPI + 压测工具	延迟/吞吐、服务化、性能瓶颈
数据管线优化	DataLoader / 多进程预处理	IO 瓶颈、GPU 利用率
Linux 排障实战	nvidia-smi/top/strace/日志	定位问题、根因分析

每个项目都留一份 "问题-定位-优化-数字" 的记录，简历上一行、面试里能聊 10 分钟。

模块五 / 5

3-5 年后，这岗位会怎么变

大模型和 Agent 越来越强
AI Infra 是被替代，还是更稀缺？

结论：不是被替代，而是更稀缺

为什么不会被替代

- ◆ 模型越强，越依赖底层算力跑得高效
- ◆ Agent/大模型本身就跑在 Infra 上，需求只增不减
- ◆ 系统级的判断和权衡，AI 短期难以替代

什么会被替代

- ◆ 纯体力的调包、写重复脚本
- ◆ 只会跑通、不懂原理的"操作工"
- ◆ AI 会成为你的工具，替代不了"懂系统的你"

真正的分水岭：你是"被 AI 替代的执行者"，还是"用 AI 放大产出的工程师"。



入行后，前 3 年这样积累

第 1 年

把系统摸熟

吃透一个方向的工具链，能独立排障，
建立"系统手感"

第 2 年

做出量化成果

能主导一个优化项，把利用率/延迟/成本做出可量化的改善

第 3 年

形成体系判断

能做技术选型和权衡，往训练/算子等硬核方向纵深

贯穿三年的一条线：持续积累"懂原理 + 会排障 + 有数字"的项目，这是你最抗替代的资产。

今天的 5 句话

1 · 是什么

让算力高效、稳定、规模化跑起来的系统层

2 · 从哪切

新人先从数据/存储、稳定性、推理服务上桌

3 · 考什么

OS、网络、并行、C++/Python、Linux 排障

4 · 练什么

小规模复现大问题，留下"问题-定位-数字"的记录

5 · 值不值得入行：值得。 门槛高、稀缺、抗替代，关键看你愿不愿意"懂系统"。

下节预告：手把手拆解第一个可写进简历的 AI Infra 项目。有问题现在就问。

